CS525 – Formal Methods for System Verification

Course Project – Phase I Evaluation Report

LLM-Driven SVA Generation with Structured Representation of Specifications and RTL

Exploring AssertionForge and Extensions for Context-Aware Formal Verification Assertion Generation

Group 2 · Project 2

Pranjal	230101077
Dhruv R Pansuriya	230101071
Shrish Uttarwar	230101108

Date: 25 March 2026

Indian Institute of Technology Guwahati

Department of Computer Science & Engineering

1. Abstract

Formal Verification (FV) is a cornerstone of reliable hardware design, and SystemVerilog Assertions (SVAs) are the primary vehicle through which modern verification engineers specify and check design correctness. Writing these assertions manually is expert-intensive and error-prone. Recent Large Language Model (LLM)-based approaches attempt to automate this process, but are limited by context constraints, ambiguous specifications, and an inability to capture the full design context.

This report presents our Phase I progress on **Project 2** of **CS525**, focused on understanding, reproducing, and extending the state-of-the-art ***AssertionForge*** framework [1]. We have successfully cloned the repository, adapted the codebase to run with the Groq API, and generated Knowledge Graphs (KGs), natural-language test plans, and final SVAs for custom hardware designs. We then survey related work in prompt compression (LLMLingua [2]), insufficiency detection (ClarifyGPT [3]), and structure-guided retrieval (STELLAR), identifying concrete extensions targeting the five problems stated in our problem set: long-context limits, agent memory, cloud communication efficiency, redundancy in multi-agent systems, and feedback-driven refinement.

2. Introduction

Assertion-Based Verification (ABV) is the dominant paradigm for ensuring that VLSI designs behave according to specification. SVAs allow engineers to express properties such as *"whenever a request is asserted, an acknowledgement must follow within three clock cycles"*, which a formal tool like Cadence Jasper can exhaustively verify. The bottleneck is that writing SVAs requires simultaneously understanding a high-level specification document (often hundreds of pages of English prose) and a low-level RTL implementation (tens of thousands of lines of Verilog).

Existing LLM-based methods such as AssertLLM [4] treat the specification as the sole information source. They miss implementation-specific details embedded in the RTL, producing assertions that are plausible but functionally incorrect. Conversely, RTL-only approaches lack the *design intent* captured in the specification. The core research challenge is:

> *How can an automated system synthesise information from ambiguous, multi-page natural-language specifications and complex RTL code to generate SVAs that are both syntactically correct and functionally meaningful — within the token constraints of modern LLMs?*

Our project addresses this challenge through five specific sub-problems defined in the course problem set (PS):

P1. Long-Context Limits — selecting a minimal, sufficient context subset.

P2. Agent Memory — accumulating verified facts across signals.[In progress]

P3. Cloud Communication Efficiency — reducing API token cost.

P4. Redundancy in Multi-Agent Systems — deduplicating overlapping assertions.[In progress]

P5. Feedback-Driven Refinement — using verifier output to improve generation.

Our Phase I report covers: study of the AssertionForge algorithm, our working implementation, and a roadmap of proposed extensions.

3. Related Work

3.1 AssertLLM

AssertLLM [4] was among the first end-to-end LLM pipelines for SVA generation from natural-language specifications. It decomposes the task into three phases: structural specification extraction, signal mapping, and assertion generation. The key limitation is that it operates *solely* on specification documents. When the specification is ambiguous or incomplete, AssertLLM generates plausible assertions that fail formal verification. It serves as the primary baseline for AssertionForge.

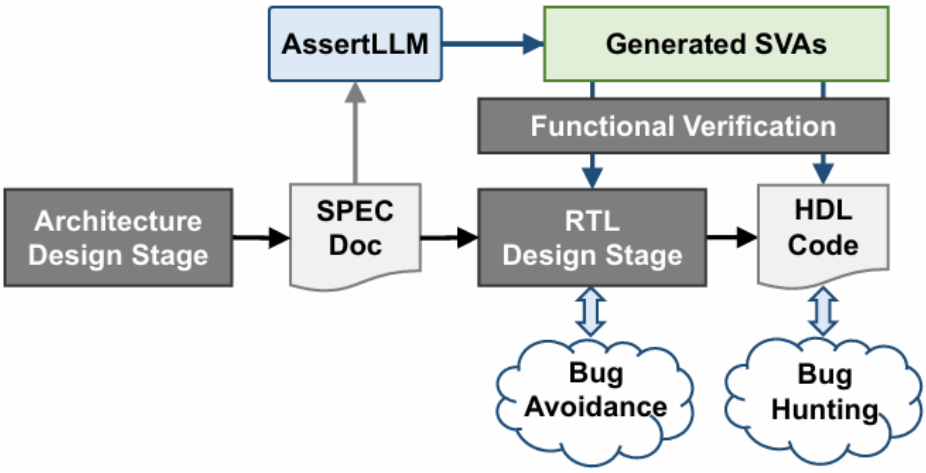

Figure 1: AssertLLM in VLSI design and verification flow. AssertLLM automatically generates SVAs from natural language specifications, facilitating functional verification for both bug avoidance and bug hunting.

3.1.1 Challenges in Understanding and Implementation

Despite its structured pipeline, reproducing or building upon AssertLLM poses several challenges:

- **Ambiguity in natural language specifications:** Translating informal and often under-specified requirements into precise formal properties is inherently difficult and error-prone.
- **Almost no implementation details and data provided:** The paper provides only a high-level description of the pipeline, with minimal insight into critical implementation aspects such as prompt design, intermediate representations, model configurations, and post-processing steps. In particular, there is no clear specification of how structural information is extracted, how signal mapping is concretely performed, or how ambiguities are resolved during assertion generation.

3.2 LLMLingua — Prompt Compression

LLMLingua [2] proposes a coarse-to-fine prompt compression framework that uses a small language model's perplexity scores to identify and remove low-information tokens from a prompt before sending it to an expensive black-box LLM. The system has three components: (1) a *Budget Controller* that allocates different compression ratios to instructions, demonstrations, and

questions; (2) an *Iterative Token-Level Prompt Compression* (ITPC) algorithm that models inter-token dependencies during compression; and (3) a *Distribution Alignment* module that fine-tunes the small model to better approximate the target LLM's distribution. The paper reports up to 20× compression with less than 1.5 percentage-point performance drop. This is directly relevant to our P1 (long-context) and P3 (cloud efficiency) sub-problems. framework that uses a small language model's perplexity scores to identify and remove low-information tokens from a prompt before sending it to an expensive black-box LLM. The system has three components: (1) a *Budget Controller* that allocates different compression ratios to instructions, demonstrations, and questions; (2) an *Iterative Token-Level Prompt Compression* (ITPC) algorithm that models inter-token dependencies during compression; and (3) a *Distribution Alignment* module that fine-tunes the small model to better approximate the target LLM's distribution. The paper reports up to 20× compression with less than 1.5 percentage-point performance drop. This is directly relevant to our P1 (long-context) and P3 (cloud efficiency) sub-problems.

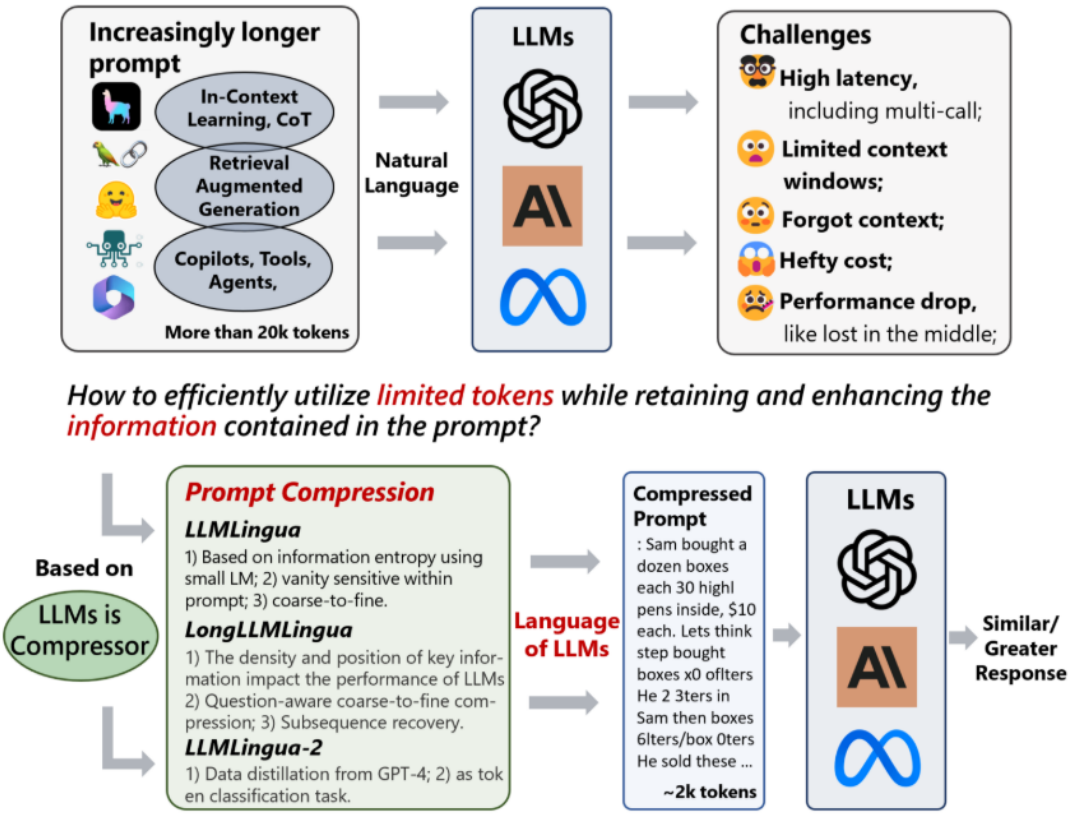

Figure 2: Why we need prompt compression.

3.2.1 Practical Challenge: Local Model Deployment

A key challenge encountered while working with LLMLingua is the requirement of running a small language model locally to compute perplexity scores for prompt compression. Although termed "small," these models still demand significant GPU/CPU memory, especially when processing long prompts or batching inputs. On resource-constrained machines, loading and running such models can lead to memory exhaustion, slow inference, or the need for aggressive optimization techniques (e.g., quantization or offloading). This creates a practical barrier to adoption, particularly when

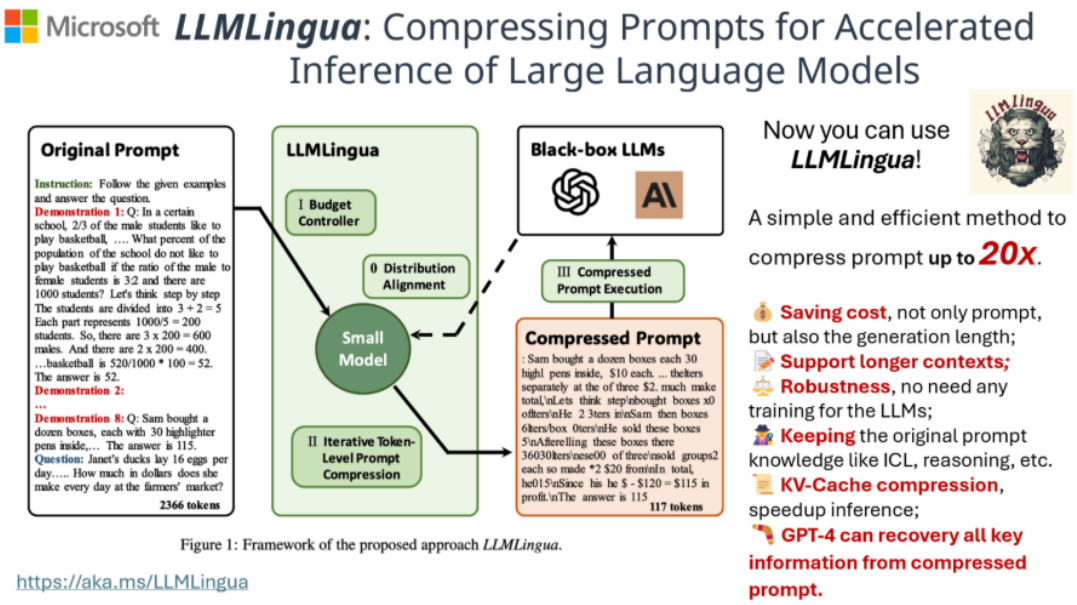

Figure 3: LLMLingua pipeline.

the goal is to reduce cost and improve efficiency, as the overhead of maintaining a local model may offset some of the intended benefits.

3.3 ClarifyGPT — Insufficiency Detection

ClarifyGPT [3] addresses the scenario where a user's requirement is ambiguous, preventing correct code generation. The system detects ambiguity by generating multiple code solutions and checking whether they produce consistent outputs on test inputs. If outputs differ, the requirement is flagged as ambiguous, and a Chain-of-Thought prompt is used to generate targeted clarifying questions. Applied to our domain: when the context assembled for signal X is insufficient to write a correct SVA, the system should *detect the gap* and *request specific missing information* rather than silently generating a wrong assertion. ClarifyGPT provides the conceptual framework; the instantiation in our setting uses RTL signal assignments as the oracle instead of user responses.

3.4 STELLAR — Structure-Guided SVA Retrieval

STELLAR is a recent framework (2025) that addresses the limitation of semantic-similarity-based retrieval for SVA generation. It represents RTL blocks as Abstract Syntax Tree (AST) structural fingerprints and retrieves structurally similar (RTL block, SVA) pairs from a knowledge base, injecting them as few-shot in-context examples. The key insight is that two RTL blocks with the same AST structure (e.g., both having a clocked always block with a case statement and a reset condition) are likely to require similar assertion patterns.

3.5 SANGAM — Monte Carlo Tree Search for Iterative SVA Generation

SANGAM applies LLM-guided Monte Carlo Tree Search (MCTS) to SVA generation from multi-modal specifications, iteratively refining assertions based on critic and formal feedback. It generates 204% more correct assertions compared to AssertLLM on I2C designs. This approach is relevant to our P5 (feedback-driven refinement) sub-problem, where verifier output is used to guide the next generation round.

4. AssertionForge: Algorithm Deep Dive

AssertionForge [1] is a two-stage, three-component pipeline. We studied the paper and source code in detail. Figure 4 illustrates the complete workflow.

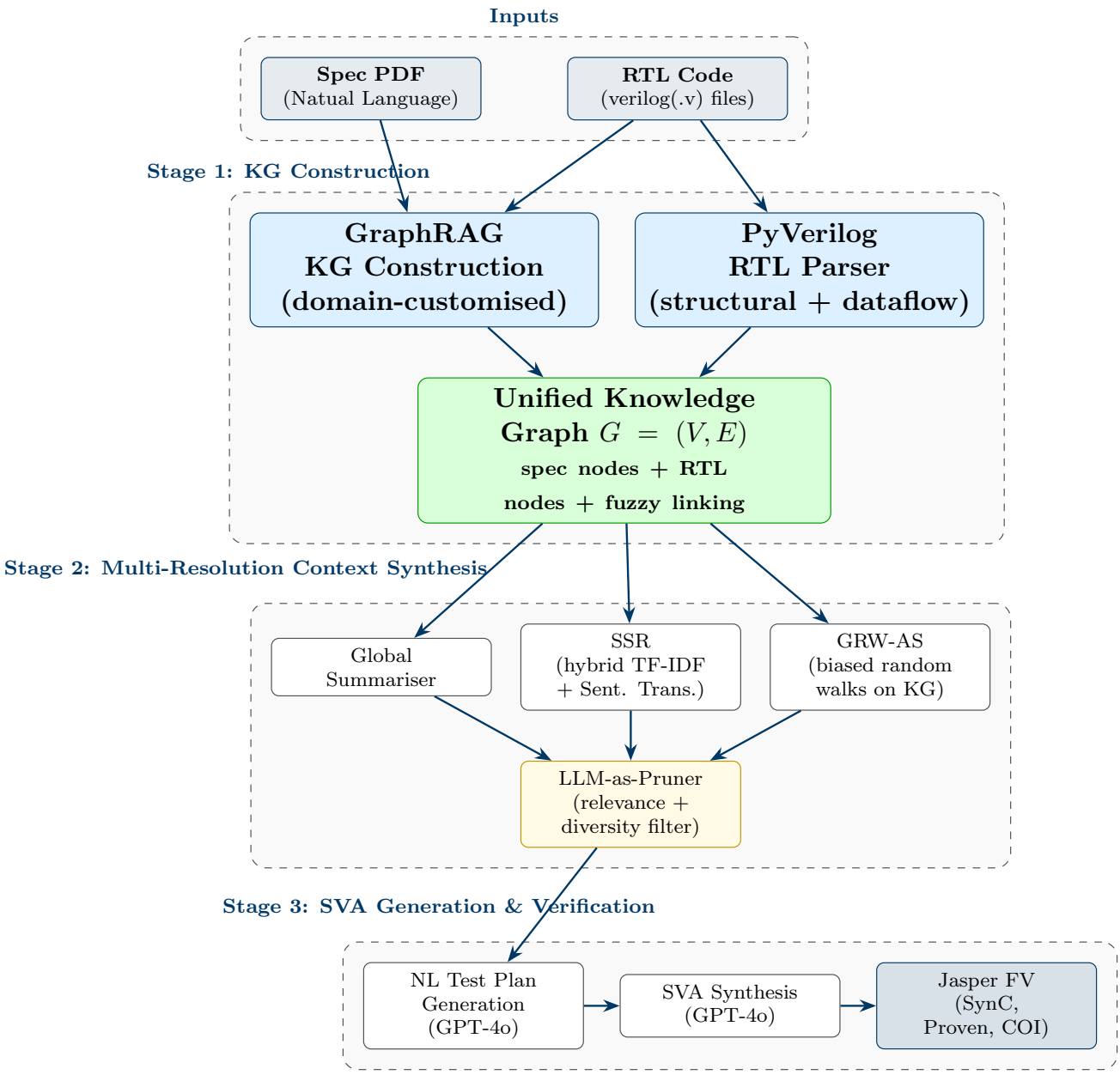

Figure 4: AssertionForge end-to-end pipeline. Arrows show data flow between components.

4.1 Stage 1: Knowledge Graph Construction

4.1.1 Specification-Side: GraphRAG with Domain-Customised Schema

AssertionForge begins by constructing an initial KG G_0 from the specification document. It leverages Microsoft GraphRAG [5], a framework that (i) chunks source text, (ii) uses an LLM to extract entities and relationships, (iii) summarises node/edge descriptions, and (iv) assembles a graph. The critical novelty is that AssertionForge replaces GraphRAG's generic entity-extraction prompt with a hardware-specific schema $\Sigma = (E_t, R_t)$:

- **Entity types E_t:** Design Specification, Module, Submodule, Signal, Port, Register, FIFO, Clock, Interrupt, Protocol, Operation, Constraint/Rule, Address, FSM.
- **Relation types R_t:** `connectsTo`, `defines`, `triggers`, `dependsOn`, `generatesInterrupt`, `operatesAt`, `transmitsData`, `HasSignal`, `UsesClock`, `DescribesOperation`, *etc.*

Node attributes include entity type, description, source chunk ID, degree, and level. Edge attributes include weight and description. The schema is discovered semi-automatically: a small set of specification excerpts is shown to the LLM, which proposes entity/relation types; a human expert then merges redundant concepts (e.g., "Reset Signal" and "Reset").

4.1.2 RTL-Side: PyVerilog Parser and KG Refinement

The initial G_0 is then refined using structural and dataflow information extracted from RTL files via PyVerilog [6]:

1. **Module structure:** AST traversal identifies `ModuleDef` nodes, port declarations (direction, width), and module instantiations.
2. **Signals and assignments:** Both continuous (`assign`) and procedural (always-block) assignments are extracted. Each assignment records the LHS target signal and the RHS dependency set.
3. **FSM detection:** Always blocks with clock sensitivity lists containing `case` statements are tagged as FSM nodes; state variable names matching patterns like `state`, `st_`, `current_state` are identified.
4. **Dataflow analysis:** PyVerilog's `VerilogDataflowAnalyzer` builds a binding graph via `getBindings()`, adding dataflow edges between signals.

4.2 Stage 2: Multi-Resolution Context Synthesis

For each target architectural signal $v_i \in V$, the system constructs up to $B = 3$ distinct prompts by combining contexts from three generators:

(i) Global Summarisation. An LLM generates five complementary summaries from the full spec + RTL: a high-level design overview (3–5 sentences), an RTL architecture summary, a comprehensive per-signal summary, a design patterns summary (FSMs, handshake protocols, pipelines), and a signal-specific description for v_i. These are generated once and cached across all signals.

(ii) Signal-Specific Retrieval (SSR). A hybrid retriever queries the specification PDF and RTL files using the signal name as the query term. It combines TF-IDF (sparse, keyword-sensitive) with Sentence Transformer embeddings (dense, semantic) at five granularity levels: 50, 100, 200, 800, 3200 tokens, with overlap ratios of 0.2 and 0.4 to address boundary fragmentation. The top 20 chunks per level are ranked by averaged cosine similarity.

(iii) Guided Random Walk with Adaptive Sampling (GRW-AS). GRW-AS is the most novel contribution of AssertionForge. Starting from the KG node corresponding to v_i, the algorithm performs 70 biased random walks with a 100-step budget each. At every step, a candidate next node c is selected with probability (Equations present in the paper). Walks terminate on reaching another architectural signal or exceeding the step budget. The resulting path is con-

verted to a natural-language description ("signal A *drives* signal B, which *is part of* module C, which *involves* verification point D") and used as a structured context fragment.

LLM-as-Pruner. The union of SSR and GRW-AS contexts is passed to a pruner LLM that scores each context on relevance to v_i, information density, and diversity complementarity, retaining the highest-ranked subset to fit within the prompt token budget.

4.3 Stage 3: SVA Generation

The final pruned contexts are assembled into dynamic prompts and fed to GPT-4o in two substeps:

1. **NL Test Plan:** The LLM first generates natural-language descriptions of properties to verify (e.g., "Verify that `TX_EMPTY` asserts when the FIFO count reaches zero").
2. **SVA Synthesis:** Each NL plan is converted into a formal SVA. The prompt includes examples of correct SVA syntax to guide the model.

Algorithm 1 summarises the complete workflow.

Algorithm 1 AssertionForge Main Workflow

Require: Specification S, RTL code R
Ensure: Generated SVAs $\mathcal{A}$

$S \leftarrow \text{PDFParser}(S)$
$G_0 \leftarrow \text{GraphRAG}(S, \Sigma)$ ▷ domain-customised KG from spec
$G \leftarrow \psi(G_0, R)$ ▷ refine with RTL via PyVerilog
$V \leftarrow \text{RTLSignalExtractor}(R)$ ▷ architectural signals
$C_g \leftarrow \text{GlobalSummariser}(S, R, V)$ ▷ cached summaries
for each signal $v_i \in V$ **do**
  $C_r \leftarrow \text{SSR}(v_i, S, R)$ ▷ hybrid RAG retrieval
  $C_k \leftarrow \text{GRW-AS}(G, v_i, V)$ ▷ Eq. **??**
  $C_p \leftarrow \text{LLMPruner}(C_g \cup C_r \cup C_k)$
  for $j = 1 \ldots B$ **do** ▷ $B = 3$ prompts per signal
    $D_{i,j} \leftarrow \Omega(C_p, v_i, j)$ ▷ dynamic prompt assembly
    $P_{i,j} \leftarrow \text{LLM}_{\text{plan}}(D_{i,j})$ ▷ NL test plans
    $A_{i,j} \leftarrow \text{LLM}_{\text{sva}}(P_{i,j}, D_{i,j})$ ▷ SVA synthesis
  end for
end for
return $\bigcup_{i,j} A_{i,j}$

4.4 Why the KG Approach is Superior

The KG provides three capabilities that flat RAG cannot:

(i) **Cross-document traceability.** A single node can link a specification sentence to the RTL signal it describes, something no chunk-level retrieval can do reliably.
(ii) **Causal chain discovery.** GRW-AS surfaces paths like `TX_EMPTY` → `apb (module)` → `PRESETn` → `reset_behavior`, which reveal that `TX_EMPTY`'s reset is governed by an active-low global reset—information that would never appear in a single 200-token chunk.

(iii) **Compression resistance.** KG path descriptions are already dense and low-redundancy. Every token carries unique structural information, so they survive token-level compression better than verbose specification prose.

5. Implementation Progress

5.1 Repository Setup and Adaptation

To establish our baseline and thoroughly understand the state-of-the-art, we cloned the official AssertionForge repository (https://github.com/NVlabs/AssertionForge) and undertook a comprehensive teardown of its architecture.

- **Codebase Reverse-Engineering and Environment Setup:** We traced the end-to-end execution flow across the entire pipeline. This involved dissecting `config.py` and `main.py` to map the configuration schemas, analyzing the PyVerilog-based RTL structural parser, and demystifying the GraphRAG-based KG builder. We tracked how intermediate data structures are passed between the Signal-Specific Retrieval (SSR), Guided Random Walk (GRW-AS), and the prompt assembler. Resolving environment dependency conflicts between hardware parsers and graph processing libraries was a necessary first step to achieve a stable build and it took a lot of time to resolve all the errors!

- **Custom LLM Backend Migration (OpenAI to Groq API):** The original framework hardcoded OpenAI's GPT-4o for both context pruning and SVA synthesis. To make our experiments cost-effective and scalable, we engineered a custom LLM wrapper to migrate the backend to the **Groq API**, specifically targeting the `llama-3.3-70b-versatile` model. This was a non-trivial transition: it required rewriting the API invocation layers, adapting the system prompts to suit LLaMA's specific instruction-following nuances, and implementing robust retry/throttling logic to gracefully handle the strict rate limits of the Groq free tier without crashing the multi-stage pipeline.

- **End-to-End Pipeline Validation on Custom Design:** Rather than relying solely on prepackaged benchmark scripts, we designed a custom, targeted Verilog module paired with a corresponding natural-language specification PDF to stress-test the adapted system. We iteratively debugged and executed **Stage 1 (KG Construction)**, verifying that the fuzzy linking between the specification nodes and RTL signals functioned correctly under the new LLM backend. We then successfully ran **Stage 2**, ensuring the multi-resolution context synthesis effectively pruned the prompts.

- **Artifact Generation and Sanity Checking:** This setup effort culminated in the successful, automated generation of all pipeline artifacts. We reviewed the intermediate natural-language test plans for logical soundness, and generated the final SystemVerilog Assertions. This working, localized pipeline now serves as the robust foundation for our proposed Phase II extensions.

5.2 Generated Outputs

The following screenshots show the system running end-to-end on our custom design. Figure 5 shows a generated natural-language test plan, and Figure 6 shows the corresponding SystemVerilog Assertions.

nl_plans.txt U

AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > nl_plans.txt

```
Signal PADDR:
Plan 1: PADDR: that when PADDR is within the range of 0 to 31, in the next cycle PADDR will remain within the valid address range.
Plan 2: PADDR: that the address signal PADDR has a value between 0 and 15, inclusive, 2 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal PCL
Plan 3: PADDR: that if the address signal PADDR is within the range of 16 to 23 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0 to 31 inclus
Plan 4: PADDR: that the address signal PADDR is within the range 0 to 7 inclusive, starting from three clock cycles after the reset signal PRESETn becomes deasserted, as indicated by
Plan 5: PADDR: that when PADDR is within the range of 24 to 31, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE.
Plan 6: PADDR: that the address signal PADDR has a value between 8 and 15, inclusive, 4 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal PCL
Plan 7: PADDR: that if the address signal PADDR is within the range of 0 to 7 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0 to 15 inclusiv
Plan 8: PADDR: that the address signal PADDR is within the range 16 to 23 inclusive, starting from five clock cycles after the reset signal PRESETn becomes deasserted, as indicated b
Plan 9: PADDR: that when PADDR is within the range of 8 to 15, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE and the select
Plan 10: PADDR: that the address signal PADDR has a value between 24 and 31, inclusive, 6 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal P

Signal PCLK:
Plan 11: PCLK: that when PCLK rises, if PENABLE is high and PSEL is high, then in the next clock cycle, PREADY will be high if the transfer is completed.
Plan 12: PCLK: that if PRESETn is low, then PCLK will not affect the state of PSLVERR until PRESETn becomes high.
Plan 13: PCLK: that when PCLK falls, if PWRITE is high, then in the next clock cycle, PRDATA will be invalid until PREADY becomes high.
Plan 14: PCLK: that after reset (PRESETn becomes high), PCLK will start generating clock edges, and PADDR will be stable after 2 clock cycles.
Plan 15: PCLK: that if PCLK is stable and PENABLE is high, then PSLVERR will be low if the transfer is successful.
Plan 16: PCLK: that when PCLK rises, if PSEL is low, then PENABLE will be ignored in the current clock cycle.
Plan 17: PCLK: that if PRESETn is high and PCLK is stable, then PADDR will be valid after 1 clock cycle.
Plan 18: PCLK: that when PCLK falls, if PENABLE is low, then PREADY will be low in the next clock cycle.
Plan 19: PCLK: that after 5 clock cycles of PCLK, if PWRITE is low, then PRDATA will be valid if PREADY is high.
Plan 20: PCLK: that if PCLK is not oscillating and PRESETn is high, then PSLVERR will be low after 3 clock cycles.

Signal PENABLE:
Plan 21: PENABLE: that when PENABLE is asserted and PSEL is high, PREADY will be set to 1 in the next clock cycle, as indicated by the clock signal PCLK.
Plan 22: PENABLE: that if PENABLE is low and PSEL is asserted, the state machine will move to the ACCESS phase in the next clock cycle, as determined by the clock signal PCLK.
Plan 23: PENABLE: that when PENABLE is asserted and PADDR is within the valid range, PSLVERR will be set to 0, indicating no error, as checked by the clock signal PCLK.
Plan 24: PENABLE: that if PENABLE is asserted and PWRITE is high, the write transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK and the PREAD
Plan 25: PENABLE: that when PENABLE is asserted and PSEL is high, PRDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY signal.
Plan 26: PENABLE: that if PENABLE is low and PRESETn is asserted, the state machine will remain in the IDLE state, as checked by the clock signal PCLK.
Plan 27: PENABLE: that when PENABLE is asserted and PADDR is out of range, PSLVERR will be set to 1, indicating an error, as checked by the clock signal PCLK.
Plan 28: PENABLE: that if PENABLE is asserted and PSEL is low, the state machine will not move to the ACCESS phase, as determined by the clock signal PCLK.
Plan 29: PENABLE: that when PENABLE is asserted and PWRITE is low, the read transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK and the PREAD
Plan 30: PENABLE: that if PENABLE is asserted and PSEL is high, PWDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY signal.
```

Figure 5: Generated natural-language test plans for a custom design, produced by AssertionForge running on Groq API.

property_goldmine_3.sva U

AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > tbs > property_goldmine_3.sva

```
module apb_props #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32
)(
    input  wire                  PCLK,
    input  wire                  PRESETn,
    input  wire [ADDR_WIDTH-1:0] PADDR,
    input  wire                  PSEL,
    input  wire                  PENABLE,
    input  wire                  PWRITE,
    input  wire [DATA_WIDTH-1:0] PWDATA,
    input  wire [DATA_WIDTH-1:0] PRDATA,
    input  wire                  PREADY,
    input  wire                  PSLVERR
  );

  property a3;
  @(posedge PCLK) ((PADDR >= 24) && (PADDR <= 31)) && (PENABLE) |-> ((PADDR >= 0) && (PADDR <= 31));
  endproperty
  assert_a3: assert property(a3);

  endmodule
```

Figure 6: SystemVerilog Assertions generated by the pipeline for the custom design.

5.3 Key Observations During Implementation

Running the system end-to-end revealed several practical insights:

1. **KG quality is highly schema-dependent.** Signals that are well-documented in the specification (e.g., clock and reset signals) form rich KG neighborhoods with many edges. Signals that are only mentioned incidentally in the spec produce sparse neighborhoods, leading to GRW-AS paths that are short and uninformative.
2. **The LLM pruner is the single most expensive call** per signal, consuming approximately 30–40% of total API tokens per signal due to the large number of candidate contexts it must evaluate simultaneously.

6. Proposed Extensions

Based on our study of the system and the open problems in the PS, we propose the following extensions. Figure 7 shows how they integrate into the existing AssertionForge architecture.

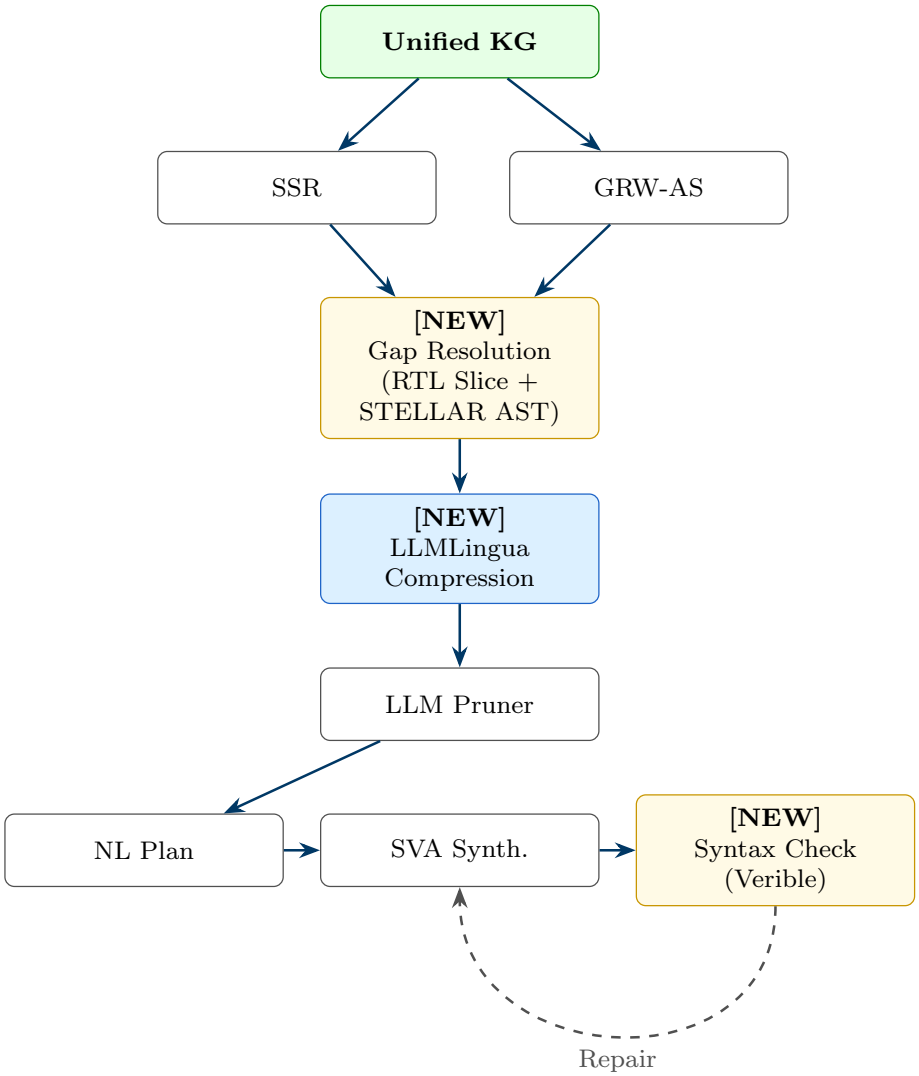

Figure 7: Extended pipeline with proposed new components (blue bordered boxes). Dashed arrow represents the P5 feedback loop.

6.1 The Missing Context Problem: A Root Cause Analysis

When a signal's Knowledge Graph (KG) neighbourhood is sparse, the generated SystemVerilog Assertions (SVAs) typically pass basic syntax checks but fail functionally because the LLM lacks the behavioral constraints necessary to write a correct property. Inspired by **ClarifyGPT's** mechanism for proactively detecting ambiguous prompts via consistency checks, we implement an early-stage "context gap" detection using a KG density threshold.

If a signal's KG degree falls below this threshold, we categorize the missing context into one of three distinct root causes:

Cause A: **KG Extraction Failure:** The signal's behavior is fully implemented in the RTL, but the GraphRAG pipeline failed to extract its relationships from the natural-language text, resulting in an artificially sparse KG neighborhood.

Cause B: **Specification Incompleteness:** The natural-language specification document is flawed and completely omits the behavioral rules or edge-case handling for the target signal. No retrieval method can find what is not documented.

Cause C: **Structural Blindness of Retrieval:** Traditional semantic or keyword search successfully retrieves text mentioning the signal by name, but entirely misses the underlying control flow (e.g., nested `if-else` conditions, FSM state transitions, or clock-domain crossings) that governs its actual hardware behavior.

AssertionForge's current Signal-Specific Retrieval (SSR) and Guided Random Walk (GRW-AS) pipelines are susceptible to all three failure modes. To resolve these gaps, we trigger two parallel, deterministic fallback approaches.

6.1.1 Approach 1: RTL Program Slicing (Addresses Causes A & B)

When specification context is either un-extracted (Cause A) or missing entirely (Cause B), the ground-truth behavioral context must be derived directly from the hardware implementation. We compute a *backward program slice* from the sparse target signal v_i within the RTL.

Starting from an assignment (e.g., `X = expr`), a backward slice systematically traverses the Abstract Syntax Tree (AST) to gather all RTL statements that influence `X`. This includes always-block sensitivity lists, FSM state guards, reset conditions, and all dependent right-hand-side input signals.

$$\text{Slice}(v_i) = \{s \in \text{RTL} \mid s \text{ strictly influences the logical value of } v_i\} \tag{1}$$

Because it is derived mechanically via PyVerilog, this slice bypasses the KG entirely. The resulting code block is highly compact (typically 20–50 lines) and is directly injected into the prompt as an unimpeachable source of behavioral truth.

6.1.2 Approach 2: STELLAR-Inspired Structure-Guided Retrieval (Addresses Cause C)

While an RTL slice provides the raw logic, LLMs often struggle to generalize low-level Verilog into a high-level formal property without strong in-context examples. Semantic similarity alone fails here because it retrieves examples based on keyword overlap rather than control-flow similarity.

To address this, we integrate the core insight from the recent **STELLAR** framework. Instead of relying on textual embeddings, STELLAR represents RTL blocks as AST structural fingerprints. Because our pipeline already utilizes PyVerilog for slicing, we can extract the AST fingerprint for the sparse signal with zero additional overhead.

We use this structural fingerprint to query a pre-compiled knowledge base of proven `(RTL block, Verified SVA)` pairs. For example, if signal X is driven by a clocked `always` block containing a `case` statement and an active-low reset, the system retrieves other distinct RTL blocks that share this exact AST topology, along with their correct SVAs. By injecting these as few-shot examples, the LLM is forced to mimic the verified property template. STELLAR demonstrates that structure-guided prompting improves syntax correctness from 84.9% to 99.8%; we leverage this as our primary fallback when specification context is degraded.

6.2 LLMLingua Compression (P1 + P3)

Once the full context is assembled (the global summary, SSR chunks, GRW-AS paths, and our new RTL slices/STELLAR examples), the resulting prompt often exceeds optimal token limits, increasing API costs and diluting the LLM's attention. To address this, we introduce **LLMLingua** compression as a final preprocessing step before the prompt is sent to GPT-4o or the Groq API.

LLMLingua utilizes a smaller, highly efficient language model to calculate the perplexity of each token, stripping away low-information words while preserving semantic integrity. We utilize a dynamic budget controller tailored for hardware verification:

- **Low Compression Budget (Keep 70–80%):** Applied to GRW-AS KG paths, RTL backward slices, and STELLAR structural examples. These elements are highly dense, structurally rigid, and carry critical causal logic that breaks if over-compressed.
- **High Compression Budget (Keep 35–40%):** Applied to raw SSR text chunks retrieved from the specification PDF. Natural language spec documents are highly verbose, contain redundant phrasing, and recover extremely well under aggressive perplexity-based compression.

This targeted compression directly resolves the cloud communication efficiency bottleneck (P3) by drastically reducing tokens per API call, while simultaneously fulfilling the long-context limit objective (P1 RQ1.1) by ensuring only the minimal, sufficient context subset reaches the final generation phase.

6.3 P5: Automated Syntax Validation and Feedback Loop

6.3.1 The IEEE 1800-2017 Parsing Bottleneck

A critical limitation in evaluating generated SVAs is that standard Verilog parsers, including the PyVerilog toolkit currently embedded in AssertionForge, do not support IEEE 1800-2017 SystemVerilog concurrent assertion syntax (e.g., `assert property (@(posedge clk) req |-> ##3 ack);`). Consequently, perfectly valid SVAs are frequently misclassified as syntax errors. To accurately measure the Syntactically Correct (#SynC) metric without relying on a heavy commercial formal verifier like Jasper, we must integrate a dedicated SystemVerilog parsing backend.

6.3.2 Tool Selection: Verible vs. Verilator

We evaluated two open-source validation backends to automate this process:

- **Verible (Google/ChipsAlliance):** A highly accessible, static syntax checker and language server designed specifically for SystemVerilog. It is purely syntactic, meaning it checks grammatical correctness without needing to resolve deep signal hierarchies.
- **Verilator (`--lint-only`):** A stricter, compilation-based static analysis tool. While it verifies syntax, it additionally enforces semantic correctness by checking for undeclared signals and bit-width mismatches.

For our Phase II implementation, we propose utilizing **Verible** as the primary syntax gatekeeper due to its low overhead, with **Verilator** serving as an optional stricter linting pass.

6.3.3 Methodology: The Automated SVA Wrapper

Syntax parsers cannot evaluate a bare assertion string in isolation. To automate the validation pipeline, we propose a dynamic wrapping mechanism:

1. **Signal Extraction:** We repurpose AssertionForge's existing PyVerilog AST parser to extract all port declarations (e.g., `input logic clk;`) for the target module.
2. **Module Wrapping:** The raw LLM-generated SVA is programmatically injected into a minimal, self-contained SystemVerilog test module alongside the extracted signal declarations.
3. **Execution & Parsing:** The wrapped module is piped into Verible/Verilator. If the return code is non-zero, the `stderr` output is parsed to extract the specific compilation error.

6.3.4 Closing the Loop (P5)

This automated syntax evaluation forms the foundation of our feedback-driven refinement loop (P5). Instead of simply discarding failed SVAs, the exact error string (e.g., "missing clocking event") alongside the RTL slice (from Approach 1) is fed back to the LLM in a structured repair prompt, allowing the model to self-correct its syntactic mistakes in a subsequent generation round.

7. Conclusion

In Phase I of this project, we successfully deconstructed and reproduced the state-of-the-art AssertionForge framework, overcoming engineering hurdles to migrate its tightly-coupled OpenAI backend to the cost-effective Groq API. Through end-to-end execution on custom hardware designs, we empirically identified the critical bottleneck of the baseline system: the "Missing Context Problem."

By conducting a formal root-cause analysis of this gap—categorizing failures into KG extraction errors, specification incompleteness, and structural retrieval blindness—we synthesized a robust extension roadmap that directly addresses all five open problems (P1–P5) defined in the problem set. Our proposed architecture leverages **ClarifyGPT-inspired** density checks to trigger deterministic **RTL program slicing** and **STELLAR-driven** AST structural retrieval. Furthermore, we mitigate cloud communication bottlenecks via **LLMLingua** token compression, and close the generation loop using an automated, **Verible/Verilator-backed** syntax validation and feedback repair mechanism.

Phase II will focus on the software integration and quantitative evaluation of these extensions. We aim to rigorously benchmark their collective impact on SVA syntactic correctness (#SynC), token efficiency, and functional accuracy, ultimately advancing the reliability of LLM-driven formal verification.

References

[1] Y. Bai, G. Bany Hamad, S. Suhaib, and H. Ren, "AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL," *arXiv:2503.19174*, 2025.
https://arxiv.org/abs/2503.19174 — https://github.com/NVlabs/AssertionForge

[2] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *arXiv:2310.05736*, 2023.
https://arxiv.org/abs/2310.05736

[3] F. Mu, L. Shi, S. Wang *et al.*, "ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification," *arXiv:2310.10996*, 2023.
https://arxiv.org/abs/2310.10996

[4] W. Fang, M. Li, M. Li *et al.*, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," *arXiv:2402.00386*, 2024.
https://arxiv.org/abs/2402.00386

[5] D. Edge, H. Trinh, N. Cheng *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, 2024.
https://arxiv.org/abs/2404.16130

[6] S. Takamaeda-Yamazaki, "Pyverilog: A Python-based hardware design processing toolkit for Verilog HDL," *Applied Reconfigurable Computing*, Springer, 2015.
https://github.com/PyHDI/Pyverilog

[7] Google/ChipsAlliance, "Verible: SystemVerilog Developer Tools," https://github.com/chipsalliance/verible

[8] "SANGAM: Monte Carlo Tree Search for Iterative SVA Generation," *arXiv:2506.13983*, 2025.
https://arxiv.org/abs/2506.13983

[9] "STELLAR: Structure-Guided LLM SVA Generation," *arXiv:2601.19903*, 2025.
https://arxiv.org/abs/2601.19903

[10] "AssertFix: Fixing SVAs using CDFG-based Context Retrieval and CoT," *arXiv:2509.23972*, 2025.
https://arxiv.org/abs/2509.23972

[11] M. Kang, M. Liu, G. Bany Hamad, S. Suhaib, and H. Ren, "FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware," *arXiv:2410.23299*, 2024.
https://arxiv.org/abs/2410.23299

[12] K. Maddala, B. Mali, and C. Karfa, "LAAG-RV: LLM Assisted Assertion Generation for RTL Design Verification," *IEEE ITC India*, 2024.
https://arxiv.org/abs/2409.15281